

Computer and Network Security

Academic Year 2025–2026

Responsible: Dr. Karim Lounis

FaceAttend

Secure Student Face Recognition System

Technical Report

Project 1: [P000] Secure student face recognition system

Team	Team 43 — G01
Members	Chouki Youcef (<i>team leader</i>) Ghanem Khalil Bachounda Essedik Abdel Rahim Kerroum Yahia Manaa Mohaned
Date	May 8, 2026

Abstract

Manual attendance in large classrooms is slow, error-prone, and trivially falsified by proxy reporting. **FaceAttend** addresses this as a full-stack web application that automates student attendance through multi-frame face recognition with layered liveness and anti-spoofing protection. Students are enrolled via a guided WebSocket session requiring three head poses; the backend validates quality, pose, and anti-spoofing in real time before committing averaged 512-dimensional embeddings from the InsightFace *buffalo_l* model. Attendance is marked through a burst-based recognition flow protected by signed challenges, two independent liveness layers, and a three-model anti-spoofing ensemble combining MiniFAS (full and quantised) and Hairymax binary 1.5.

The path to this design involved significant experimentation: training on CelebA produced poor results on webcam captures; FSFM showed better accuracy but was too heavy for real-time use; signal-based approaches including Moiré pattern detection, flicker-rate analysis, monocular depth estimation with DepthAnything, and temporal variance spikes each captured part of the problem but proved too sensitive to screen pixel density and ambient lighting to be reliable standalone signals. Extended gaze-following and multi-step challenge flows were also explored but proved too slow for classroom use. As a sobering baseline, even the university dormitory access system was bypassed by holding a photograph on a phone screen. This report details the technical pipelines, experimental path, observed model failures with images, and a roadmap for future hardening.

Contents

1	Introduction	3
2	System Architecture	3
2.1	Two-tier design	3
2.2	Technology stack	4
2.3	Storage	4
3	Core Pipelines	4
3.1	Enrollment pipeline	5
3.2	Recognition pipeline and challenge-response mechanism	5
3.3	Anti-spoofing ensemble	6
4	Anti-Spoofing Exploration	7
4.1	Dataset experiments: CelebA	7
4.2	Alternative deep model: FSFM	7
4.3	Signal-based detection approaches	8
4.4	MediaPipe depth analysis and its limitations	9
4.5	Observed MiniFAS failure modes	11
5	Recognition Flow Experiments	11
5.1	Extended challenge mechanisms	11
5.2	Finding the middle ground	12
6	Real-World Context: Deployed Systems are Vulnerable Too	12
7	Engineering Challenges	14
8	Current Limitations	15
9	Future Perspectives	15
9.1	Improved anti-spoofing and attack evaluation	15
9.2	Hardware liveness	15
9.3	Privacy controls and consent	15
9.4	Scalability and deployment	16
9.5	Admin review workflow	16
9.6	Multilingual and accessibility support	16
10	Conclusion	16

1. Introduction

Traditional classroom attendance relies on a teacher calling names, circulating a sign-in sheet, or trusting self-reporting. All three approaches share the same weaknesses: they consume lesson time, introduce transcription errors, and offer no verification that the person who signed in was actually present.

Face recognition offers an obvious path to automation, but a naive implementation is vulnerable to *presentation attacks*: an attacker can hold a photograph or a phone displaying an enrolled student’s face in front of the camera. This threat is not hypothetical. During our testing we presented a photograph on a phone screen to the university dormitory’s face recognition access system and were granted entry. Deployed commercial systems do not necessarily include liveness detection at all.

FaceAttend was designed around four principles:

- (1) **Server-owned decisions.** The backend controls every security-relevant step. The frontend is a guided camera interface, not a trusted authority.
- (2) **Multi-evidence anti-spoofing.** No single model is sufficient. Three independent ONNX models vote on every burst before any recognition decision is made.
- (3) **Auditable outcomes.** Every enrollment commit and every recognition decision is written to an immutable audit log.
- (4) **Replaceable AI boundaries.** Each AI component is isolated behind a thin provider interface so models can be upgraded without changing business-logic code.

This report covers system architecture (Section 2), core pipelines (Section 3), anti-spoofing exploration (Section 4), recognition flow experiments (Section 5), the real-world bypass context (Section 6), engineering challenges (Section 7), current limitations (Section 8), and future directions (Section 9).

2. System Architecture

2.1 Two-tier design

FaceAttend separates concerns between a browser frontend and a Python backend. The frontend captures and streams camera data; the backend owns all security decisions.

Table 1: Responsibility split between frontend and backend.

Tier	Responsibility
Frontend (React / Vite)	Camera access, frame capture, WebSocket enrollment streaming, MediaPipe liveness signals, challenge submission, workflow guidance, and UI rendering.
Backend (FastAPI / Python)	Frame quality assessment, pose classification, anti-spoofing ensemble, face detection and embedding, FAISS index search, challenge signing and verification, attendance marking, and audit logging.
Shared boundary	JWT authentication on all protected endpoints; one-time HMAC-SHA256 tokens for WebSocket sessions; signed short-lived challenge tokens for recognition bursts.

2.2 Technology stack

Table 2: Core technology choices.

Layer	Technology	Rationale
API server	FastAPI (Python 3.11)	Async I/O, automatic OpenAPI docs, clean dependency injection.
ORM	SQLAlchemy 2.0 + Alembic	Explicit schema migrations, async session support.
Database	SQLite (dev) / PostgreSQL	Zero-config local runs; clear upgrade path to production.
Face models	InsightFace <i>buffalo_l</i>	512-dimensional embeddings; LFW accuracy 99.83 %.
Anti-spoof	MiniFAS full + quantised, Hairymax 1.5	Ensemble of three independent ONNX models.
Vector search	FAISS (inner product)	Sub-millisecond search over per-group normalised embeddings.
Frontend	React 18 + Vite + Tailwind	Fast development iteration; self-signed TLS for camera access.
Liveness	MediaPipe Face Landmarker v2	In-browser landmark stream; no server round-trip per frame.

2.3 Storage

All persistent state is held in the relational database. Key tables are listed below.

Table 3: Principal database tables.

Table	Content
<code>users</code>	Accounts with bcrypt-hashed passwords; roles admin or teacher.
<code>organizations</code>	Multi-tenant root; all resources are org-scoped.
<code>students</code>	Non-account student records with an enrollment status flag.
<code>face_embeddings</code>	One averaged 512-dim vector per student; optional AES-256 encryption.
<code>enrollment_sessions</code>	Server-owned state machine (pending, active, ready, committed, failed).
<code>attendance_sessions</code>	Open or closed class sessions owned by a group.
<code>attendance_records</code>	Per-student status: present, absent, self-reported, or rejected.
<code>recognition_audit_logs</code>	Every recognition decision with confidence, liveness signals, and reason.

Face embeddings may be stored as plain JSON vectors or encrypted with AES-256-GCM, with support for rolling key rotation.

3. Core Pipelines

Figure 1 shows the two main runtime flows side by side.

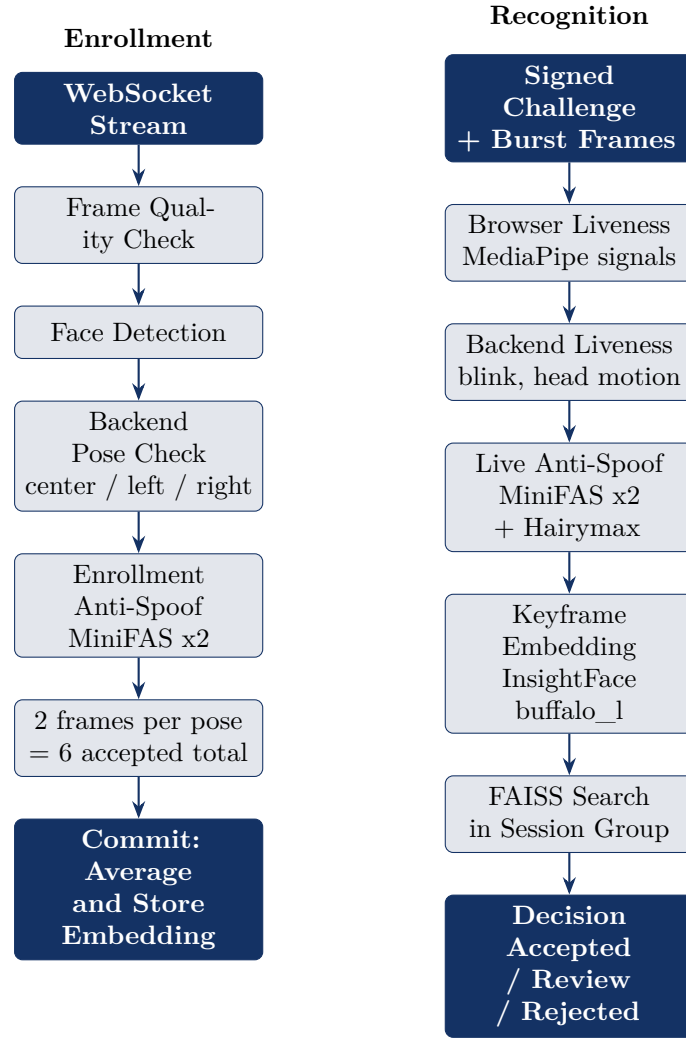


Figure 1: Enrollment (left) and recognition (right) pipelines. Both are fully server-owned; the frontend streams raw camera frames and cannot influence any security decision.

3.1 Enrollment pipeline

A teacher creates an enrollment session for a student. The backend returns a one-time HMAC-SHA256 WebSocket token (120-second TTL) and the required poses: center, left, and right, with order randomised after center.

The frontend opens the WebSocket and streams at 3 fps. For each sampled frame the backend runs five checks in order: whole-frame quality (blur, brightness, contrast); face detection (exactly one face, size and centering constraints); aligned crop quality; backend pose classification from five-point InsightFace landmarks; and enrollment anti-spoofing using both MiniFAS variants independently. A frame is rejected immediately if any check fails.

When two frames per pose (six total) are accepted, the session becomes ready and the frontend commits. On commit, embeddings are computed for all accepted frames, averaged, and L2-normalised before being stored. A duplicate-face check (cosine similarity above 0.92 against all organisation-owned embeddings) blocks re-enrollment of an already-enrolled identity. Temporary frame payloads are deleted on commit or failure.

3.2 Recognition pipeline and challenge-response mechanism

Figure 2 shows the recognition flow in detail, including the signed challenge handshake.

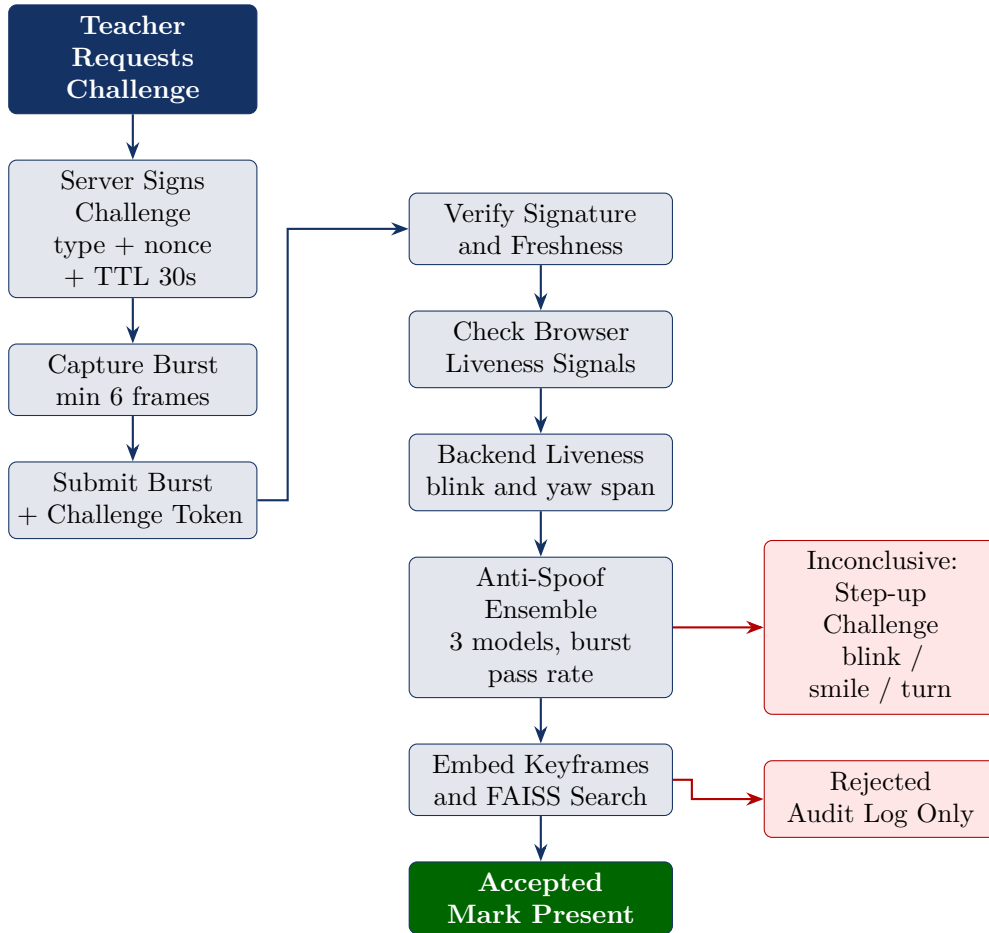


Figure 2: Recognition challenge-response flow. A server-signed token with a 30-second TTL prevents burst replay. Evidence that is ambiguous rather than clearly fake triggers a step-up challenge rather than an immediate rejection.

Confidence thresholds classify outcomes: accepted (0.85 and above), review (0.75 to 0.84), or rejected (below 0.75). A 15-minute cooldown prevents a student from being marked present twice within the same session.

3.3 Anti-spoofing ensemble

Table 4: Anti-spoofing model ensemble.

Model	Crop strategy	Public benchmark	Used in
MiniFAS full	Raw bbox + 20% padding	CASIA-MFSD EER $\approx 2.5\%$	Enroll + Live
MiniFAS quantised	Same (int8 for speed)	Near-identical to full	Enroll + Live
Hairymax 1.5	Expanded square crop, letterboxed	Competitive on Oulu-NPU	Live only

For recognition, all three models must participate. A burst frame passes only when the configured primary model and all required candidates agree the face is live. The burst-level decision aggregates the frame pass rate. Consistent multi-model failure indicates an attack. Ambiguous evidence triggers a signed step-up challenge.

4. Anti-Spoofing Exploration

Reaching the three-model ensemble required substantial experimentation across datasets, alternative neural models, and physical signal-based detection approaches. The following sections document each attempt and the reason it was insufficient on its own.

4.1 Dataset experiments: CelebA

Early in the project we trained and evaluated face recognition models using the CelebA dataset, a large-scale collection of celebrity photographs. CelebA images are high-quality, well-lit, and professionally captured. Our deployment context is standard laptop webcams under varying office lighting, with students at typical classroom distances. That environment is fundamentally different.

Models evaluated on CelebA produced poor results on webcam captures. Embeddings computed from CelebA-optimised pipelines clustered differently from webcam captures, leading to high false-reject rates for enrolled students. In short, benchmark accuracy on a clean celebrity dataset does not transfer to consumer webcam conditions, and enrollment data must come from the same capture environment as the deployment.

4.2 Alternative deep model: FSFM

We evaluated FSFM (Face Spoofing Foundation Model), a four-class detector that distinguishes real, deepfake, diffusion-generated, and presentation-attack faces. On our test images FSFM correctly identified screen-based presentation attacks with 90 to 92% confidence and classified real faces as bonafide at 83 to 89% confidence. Figure 3 shows an example result.

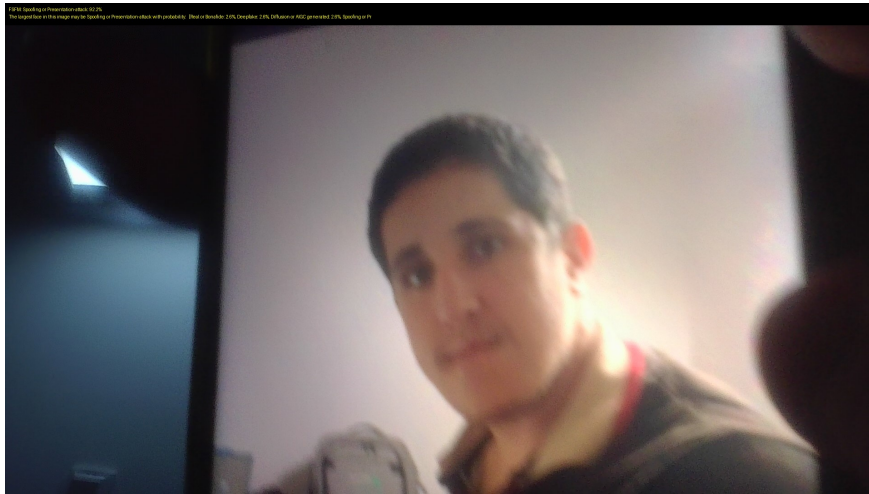


Figure 3: FSFM correctly labels a phone-screen presentation attack as “Spoofing or Presentation-attack: 92.2%”. Despite better accuracy than MiniFAS on this class of attack, FSFM required a remote Gradio inference server, failed on several test images with upstream exceptions, and was too slow for real-time burst processing.

Despite better accuracy, FSFM was not integrated for three reasons: it depends on a Gradio-based inference server, it returned exceptions rather than scores on a significant fraction of input images, and its latency is incompatible with a real-time burst pipeline. FSFM showed what a more capable model could achieve, but the integration cost was not justified for a real-time burst pipeline.

4.3 Signal-based detection approaches

Beyond dedicated neural models, we investigated four signal-based techniques. Each of them targets a real physical difference between a live face and a 2D attack, but none proved reliable enough on its own.

Moiré pattern detection. When a camera captures a screen, the sensor’s pixel grid interferes with the screen’s pixel grid and produces a regular interference pattern known as Moiré. Live skin has no such pattern. Detection operates in the frequency domain: a Fast Fourier Transform of the face crop reveals periodic high-frequency energy peaks when a screen is the source. Figure 4 illustrates the physical difference between the two cases.

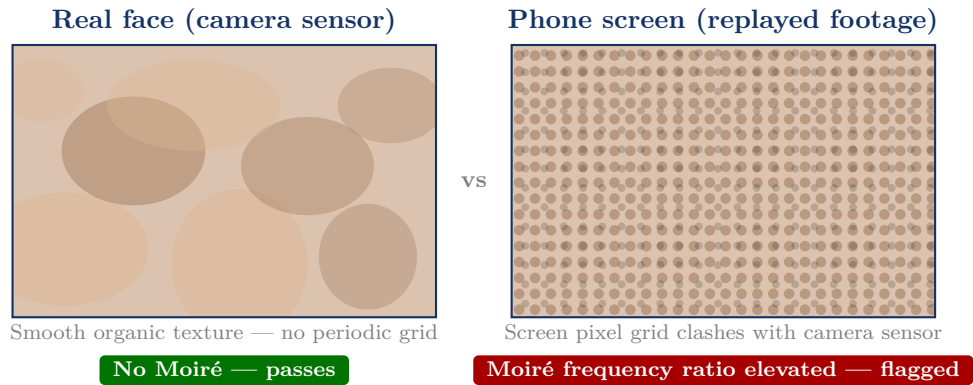
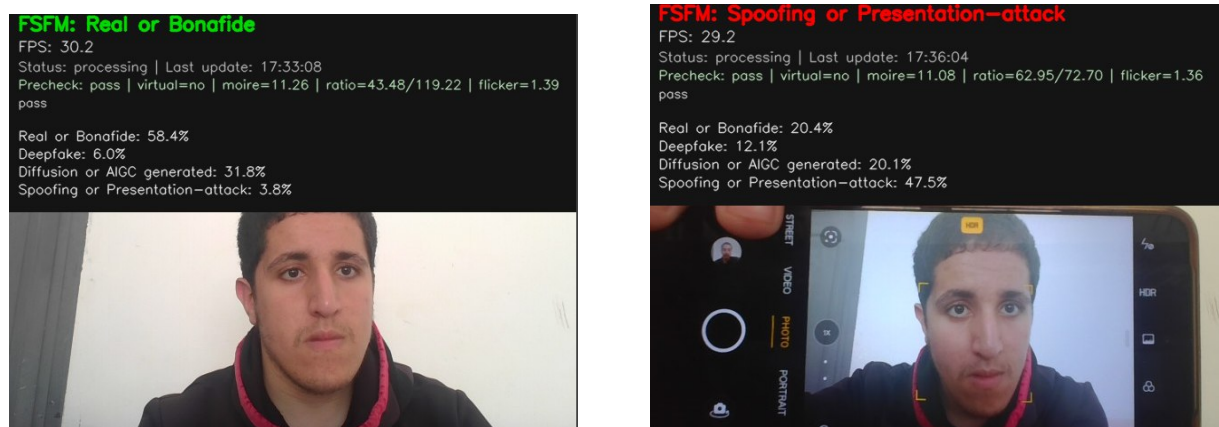


Figure 4: Concept illustration of the Moiré detection approach. A real face (left) shows organic, aperiodic skin texture. A phone screen (right) displays two overlapping regular pixel grids (the screen’s and the camera’s), which create periodic interference patterns. These patterns raise the high-frequency energy ratio in the Fourier spectrum.

Figure 5 shows our live tests with FSFM’s Moiré ratio metric.



(a) Real face: moire=11.26, frequency ratio = $43.48/119.22 \approx 0.36$. FSFM classifies as Real or Bonafide (58.4%).

(b) Phone screen attack: moire=11.08, frequency ratio = $62.95/72.70 \approx 0.87$. The ratio is noticeably higher. FSFM classifies as Spoofing (47.5%).

Figure 5: Moiré metrics recorded during live tests. The raw Moiré score (11.26 vs 11.08) is nearly identical between real and fake, but the frequency band ratio (0.36 vs 0.87) is substantially higher for the screen attack, showing that the periodic interference raises energy in specific frequency bands.

While the ratio does differ, this approach proved unreliable in practice. Modern AMOLED screens with pixel densities above 400 PPI produce Moiré patterns below the resolution threshold

of a standard 1080p webcam at arm’s length. The distinction between the two ratios (0.36 vs 0.87) was clear only for the specific phone used in the test. On other devices and under different lighting conditions, the ratios overlapped with real-face values, making a fixed threshold unusable.

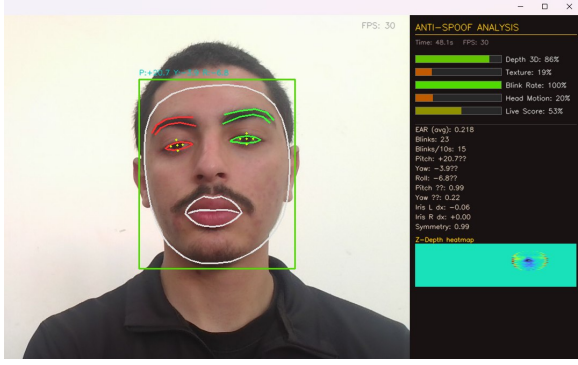
Flicker-rate detection. Screens refresh at a fixed rate (typically 60 or 120 Hz). A camera capturing a screen at a mismatched shutter rate records brightness oscillation or banding across the face crop. Analysing temporal brightness variance or applying a Fourier transform to per-frame mean intensity can reveal this periodic signal. In practice, modern screens with high and dynamic refresh rates, combined with cameras that continuously adapt exposure, largely suppress the effect. Results were inconsistent across different phones and room lighting conditions.

Monocular depth estimation (DepthAnything). A real face is three-dimensional: the nose protrudes, the eyes are recessed, the face curves. A photograph or screen is geometrically flat. We tested DepthAnything to detect this difference. In practice, monocular depth estimation for close-up faces is fundamentally ambiguous: the model infers 3D structure from 2D shading cues, and a high-quality photograph of a face contains nearly identical cues. The estimated depth map for a face displayed on a screen looked nearly the same as for a real face at the same distance, making the approach unreliable as a liveness signal.

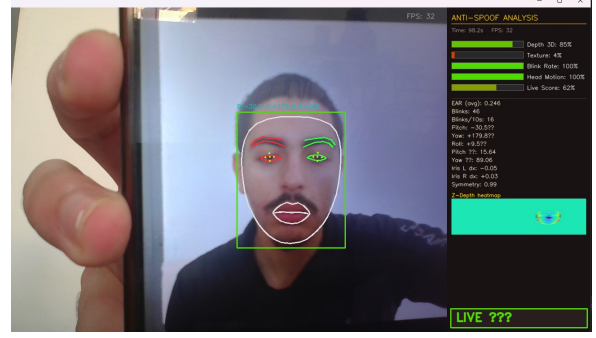
Temporal variance spikes. A static photograph in front of a camera produces near-zero temporal variance; a live face shows micro-movements, subtle expression changes, and natural head sway. Analysing pixel intensity standard deviation within the face crop across a burst should distinguish the two. The approach partially worked against static print attacks but failed against video-replay attacks (a phone playing a recorded video of the enrolled student), which produce natural temporal variance. It also triggered false rejects when a legitimate student sat unusually still, or when camera noise dominated the variance signal under low light.

4.4 MediaPipe depth analysis and its limitations

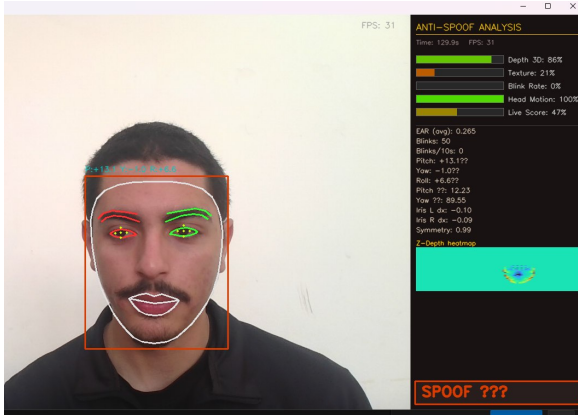
The signal-based experiments above were complemented by a dedicated anti-spoof analysis tool built on MediaPipe Face Landmarker. The tool extracts per-frame depth metrics (Depth 3D score, texture score, blink rate, head motion, and a composite live score) and overlays them on the webcam feed alongside a Z-depth heatmap derived from MediaPipe’s 3D face mesh. The intention was to determine whether the estimated depth of the face region changes meaningfully between a real face and a phone screen displaying that face. Figure 6 shows three captures: a real face with no clear decision, a phone screen attack that the system labels live, and a real face that the system labels as a spoof.



(a) Real face. Depth 3D: 86%, Live Score: 53%. System is uncertain.



(b) Phone screen attack, labeled LIVE. Depth 3D: 85%, Live Score: 62%. The attack scores higher than the real face. Green “LIVE ???” verdict shown.



(c) Real face, labeled SPOOF. Depth 3D: 86%, Live Score: 47%. A genuine student is rejected. Orange “SPOOF ???” verdict shown.

Case	Depth	Texture	Score
Real face	86%	19%	53%
Phone screen	85%	4%	62%
Real face (FP)	86%	21%	47%

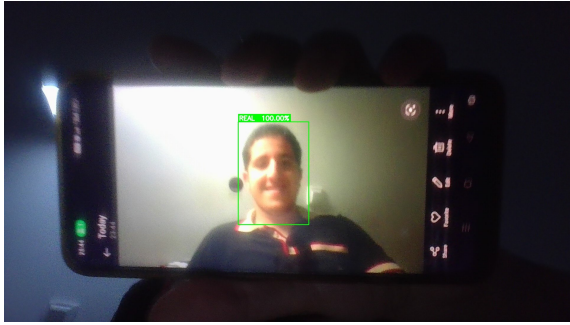
Key observation: The Depth 3D score is **virtually identical** across all three cases (85–86%), confirming that MediaPipe’s monocular depth estimation cannot separate a real face from a screen displaying one. The phone screen actually achieves a *higher* Live Score (62%) than either real face (53%, 47%), demonstrating that the composite metric can be systematically misleading. Texture score (4% vs 19–21%) is a better discriminator, but it is weighted too lightly to overcome the Depth 3D and Head Motion signals.

Figure 6: MediaPipe anti-spoof analysis across three cases. Depth 3D values are nearly constant regardless of whether the input is a live face or a presentation attack, exposing the fundamental limitation of monocular landmark-based depth as a liveness signal.

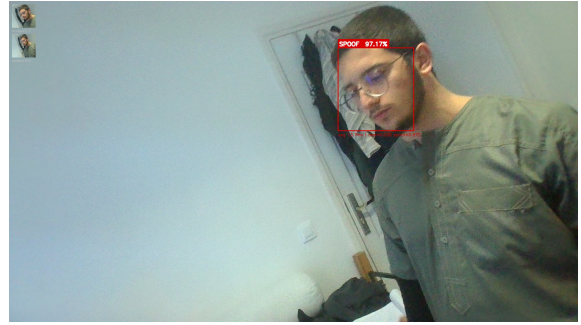
These results reinforce the DepthAnything finding: software-derived depth from a single RGB camera cannot detect the flatness of a 2D attack because the model has no true geometric information. The face landmarks it tracks appear at plausible 3D positions whether the face is real or rendered on a screen, because the underlying geometry inference relies on the same 2D image cues in both cases. The Z-depth heatmaps in all three captures show similar small, localised depth variation at the nose bridge, the only region where the model infers any meaningful depth relief, which confirms that the signal lacks discriminative power against high-quality screen attacks.

4.5 Observed MiniFAS failure modes

Even after settling on MiniFAS as the primary model, we observed significant inconsistency. Figure 7 shows two representative failure cases.



(a) A phone screen displaying a face: MiniFAS outputs **REAL 100%**. The phone UI, screen bezel, and hand are all visible, yet the model is fully confident the face is live. This is a complete false accept.



(b) A real student looking downward at something: MiniFAS outputs **SPOOF 97%**. The downward head angle produces a crop that resembles known spoof patterns in the training data, causing a confident false reject.

Figure 7: MiniFAS failure cases. Left: false accept on a high-quality phone screen attack. Right: false reject of a real face at a non-frontal angle. Both errors are near-maximum confidence, showing that the model’s score is not well-calibrated on out-of-distribution inputs.

The false accept in Figure 7a is the more serious failure: a well-lit phone screen showing an enrolled student’s face completely fools MiniFAS regardless of threshold tuning. The false reject in Figure 7b shows the opposite problem: a real user is blocked with maximum confidence due to pose.

These failures are the direct reason we moved to an ensemble. A single-model threshold tuned tight enough to block most attacks also rejects too many real users. Loosen it to let real users through, and attacks start slipping past. The ensemble breaks this dead-end: when two independent models disagree about a face, the system escalates to a step-up challenge rather than committing to a hard decision.

5. Recognition Flow Experiments

5.1 Extended challenge mechanisms

Beyond the standard blink, smile, and head-turn challenges, we explored two approaches aimed at making replay attacks harder.

Random multi-step challenges. The idea was to sequence multiple challenges in a randomised order so that a pre-recorded video could not pass all three in the correct sequence. We implemented a prototype and tested it with several classmates. Total session time increased significantly: collecting three verified challenge responses added 10 to 15 seconds per recognition attempt. For a classroom with 30 students this would make live sessions impractically long. The approach also introduced additional failure points: a student who correctly completed two challenges and failed the third due to a brief lighting change had to restart the entire sequence.

Gaze and dot-following challenge. A dot moving across the screen asks the user to follow it with their eyes. A static photograph cannot move its gaze; a pre-recorded video would not match the correct dot path. We implemented a dot-following widget using MediaPipe iris landmark

tracking. In practice, reliable iris tracking requires good lighting, the user at a specific distance, and a nearly frontal face. False reject rates were high on normal webcams: students with glasses, strong backlighting, or webcams with motion compression had their gaze estimated incorrectly, causing them to fail even when genuinely following the dot. A successful gaze test also took approximately 5 to 8 seconds, roughly tripling the total session time.

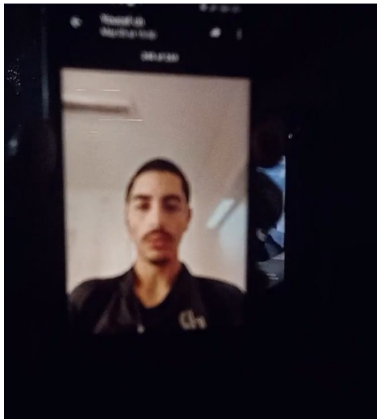
5.2 Finding the middle ground

After testing both extended mechanisms, we concluded that the added security did not justify the session-time cost and failure rate for a classroom setting. Most attack risk is already addressed by the anti-spoofing ensemble combined with the signed challenge: a pre-recorded burst fails because the challenge nonce will not match, and a real-time phone screen attack will be caught by at least one ensemble model most of the time.

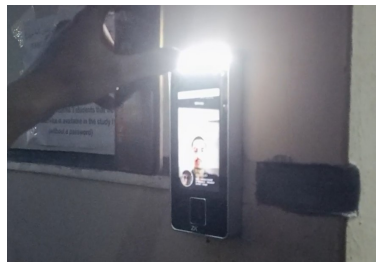
The final design uses a single signed challenge (blink, smile, or head turn), a burst of at least six frames evaluated by the full ensemble, and a step-up challenge only when evidence is genuinely ambiguous. This keeps a typical recognition attempt under five seconds from challenge request to attendance confirmation while maintaining meaningful spoofing resistance.

6. Real-World Context: Deployed Systems are Vulnerable Too

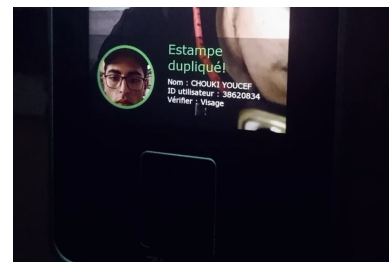
A common assumption is that commercially deployed face recognition systems are more secure than a student project. Our experience during testing challenges this assumption. The university dormitory uses a face recognition system for building entry. During a team test session, one member held a phone screen displaying a photograph of another team member in front of the dormitory camera. The system accepted it and unlocked the door.



(a) The phone screen held in front of the dormitory camera. No liveness challenge was issued.



(b) The photograph used in the attack: a still image of an enrolled team member displayed on a phone screen.



(c) Attack succeeded: the dormitory system granted access based on the photograph alone, with no liveness or anti-spoofing check.

Figure 8: Dormitory face recognition bypass demonstration. A static photograph displayed on a phone screen was sufficient to unlock the door, confirming that the deployed system performs no liveness detection.

The takeaway is simple: face recognition systems in active deployment do not necessarily include any liveness detection. FaceAttend is harder to bypass than that system. The three-model ensemble, signed challenges, and burst evaluation make a simple static photograph unlikely to succeed. However, as Figure 7a shows, a high-quality phone screen displaying a live-looking face can still fully fool MiniFAS. The distance between “better than the dormitory system” and

“resistant to a determined attacker” is where the remaining work lies.

7. Engineering Challenges

Table 5: Engineering challenges encountered and how they were addressed.

Challenge	Approach
Model latency on CPU	InsightFace and the three ONNX anti-spoof models are too slow for interactive use on CPU-only hardware. The backend selects ONNX Runtime execution providers at startup (CUDA, then OpenVINO, then CPU). A discrete GPU is required for practical deployment; the system is impractical on CPU-only machines at production frame rates.
Camera quality heterogeneity	Webcam quality varies widely between laptops. Low-resolution or compressed streams raised frame rejection rates during enrollment and recognition. Laplacian quality thresholds were tuned to balance rejection sensitivity against usability on common hardware.
Anti-spoofing model inconsistency	Both MiniFAS and Hairymax produce inconsistent verdicts under unusual lighting or high-quality screen attacks. A single model either over-rejects real users or allows attacks. The three-model ensemble with graduated step-up challenges reduces both failure modes, though neither is eliminated entirely.
Browser camera security requirements	<code>getUserMedia()</code> requires HTTPS. The Vite development server uses self-signed TLS certificates so the frontend can access the camera while proxying API and WebSocket traffic to the backend.
WebSocket token security	Standard JWT cannot be passed safely in query strings. Enrollment WebSockets use one-time HMAC-SHA256 tokens consumed on first use with a 120-second TTL, preventing theft via URL inspection or server logs.
Software-only liveness ceiling	Consumer laptops lack depth sensors. The system relies entirely on software liveness. A convincing 3D mask cannot be ruled out by software alone. This is a hardware constraint, not a software problem.
Backend-owned enrollment state	The original REST prototype let the client select frames. A server-side WebSocket state machine was introduced so that pose order, frame count, quality, and anti-spoofing are all enforced by the backend and cannot be bypassed by a modified client.
Signal techniques were environment-dependent	Moiré detection, flicker analysis, DepthAnything depth estimation, and temporal variance spikes each failed to generalise across different screen PPI values and lighting conditions. No single physical signal was reliable enough to replace a trained model.

8. Current Limitations

Table 6: Known limitations of the current system.

Limitation	Detail
High-quality screen attacks not reliably blocked	As shown in Figure 7a, a high-resolution phone screen displaying a live-looking face can produce a complete false accept from MiniFAS. The ensemble reduces this risk but does not eliminate it.
No depth sensing	The system uses only RGB cameras. A convincing 3D mask or high-resolution silicone face replica could bypass software liveness. Mitigating this requires structured-light or infrared hardware not present on standard laptops.
Lighting and camera sensitivity	Recognition degrades under low light, strong backlighting, or heavy webcam compression. No in-app exposure guidance is currently provided to the user.
GPU requirement	The full pipeline (InsightFace plus three ONNX anti-spoof models) requires a discrete GPU for interactive latency. CPU-only machines experience multi-second processing times per burst, making live sessions impractical.
No large-scale validation	Testing involved a small group of classmates. No formal false accept or false reject evaluation over a large independent dataset has been conducted.
SQLite in development	The default database is SQLite. Production deployment requires PostgreSQL with proper secrets management.
Public self check-in has no face verification	The public endpoint records attendance using only a registration number. It must not be treated as a security control.
Privacy and biometric data handling	Face embeddings are biometric data. Optional AES-256 embedding encryption is available, but retention policies, consent workflows, and deletion guarantees have not yet been implemented.

9. Future Perspectives

9.1 Improved anti-spoofing and attack evaluation

The most impactful near-term improvement is a rigorous evaluation of the ensemble on a larger, labelled attack dataset covering printed photographs, phone and monitor displays at various PPI levels, and 3D masks. Threshold and pass-rate parameters should be tuned empirically rather than heuristically. Fine-tuning MiniFAS on locally collected webcam data would likely close the gap on screen-based attacks significantly.

9.2 Hardware liveness

Integrating a depth camera would allow a direct 3D check at enrollment and recognition time. The backend’s AI provider boundary was designed to accommodate additional sensors; a depth provider implementing the existing interface would require no changes to business-logic code.

9.3 Privacy controls and consent

A production deployment must implement student consent collection before enrollment, configurable embedding retention periods, and a right-to-deletion flow. The optional AES-256 encryption layer is the technical foundation; a policy and workflow layer above it is the missing piece.

9.4 Scalability and deployment

Migrating to PostgreSQL, adding a Celery task queue for asynchronous report generation, and deploying behind a production ASGI server would bring FaceAttend to a state suitable for a real institutional rollout.

9.5 Admin review workflow

Recognition attempts in the review band (0.75 to 0.84) are currently surfaced to the teacher without enforcing a review action. A formal admin-review queue with timeout escalation would reduce ambiguity and improve attendance record reliability.

9.6 Multilingual and accessibility support

The frontend is English-only. Internationalisation and screen-reader accessibility improvements would widen the deployable audience. Improved in-app guidance for poor lighting and angle errors would directly reduce enrollment and recognition failure rates for non-technical users.

10. Conclusion

FaceAttend shows that a practical, security-conscious attendance system can be built around face recognition using standard web technologies and open-source AI models. Moving all security-sensitive steps into a server-owned pipeline was the key architectural choice: it ensures the system cannot be trivially bypassed by a modified client. The three-model anti-spoofing ensemble with a graduated challenge mechanism handles the security-usability trade-off better than any single-model approach we tested.

The experimental path turned out to be as valuable as the final product. CelebA did not transfer to webcam conditions. FSFM showed that more capable models exist, but the integration cost was not justified here. Signal-based techniques (Moiré, flicker, depth, variance) were sound in theory but unreliable in practice across different screen hardware and lighting conditions. Extended challenge flows added security at a session-time cost that made them impractical for a classroom. The dormitory bypass confirmed that even deployed commercial systems can lack basic liveness protection, which is the problem this project set out to address.

One thing the project makes clear is that software-only liveness detection has a ceiling. Without depth sensing, a sufficiently high-quality screen-based attack will eventually produce a false accept, as Figure 7a shows. This is a hardware constraint, not a design flaw. Institutions considering deployment should treat it as a risk to manage through policy (supervised sessions, random manual spot-checks) alongside the technical controls already in place.